# FIAP
# RELATÓRIO TÉCNICO
# ETAPA 3

**Sistema A.D.P.**

Análise, Decisão e Previsão
Colônia de Marte

Henrique Floriano Alcantara
Nicolas França
Joao Pedro Candido Souza
Katia Regina Bispo
Yago Souza Araujo

Ciência da Computação | 2026

## 1. Introdução

A Etapa 3 da missão Aurora propõe um sistema integrado para apoiar a operação de uma colônia em Marte. O desafio reúne organização de dados, decisões por condições lógicas, análise energética e uma previsão simples baseada em dados históricos. Desenvolvemos o Sistema A.D.P. (Análise, Decisão e Previsão) em Python para reunir essas tarefas em um único programa de terminal.

O sistema recebe informações sobre energia disponível, geração solar, geração eólica, consumo e velocidade do vento. A partir desses valores, apresenta alertas ou sugestões, aplica regras de economia que alteram o estado simulado dos sistemas e estima a geração eólica com uma regressão linear. É um protótipo didático: não recebe dados de sensores reais nem controla equipamentos físicos.

## 2. Objetivo e escopo

O objetivo foi construir um programa funcional que organize informações da colônia e transforme entradas numéricas em respostas fáceis de interpretar. A proposta também permite praticar funções, estruturas condicionais, laços de repetição, tratamento de entradas inválidas e um modelo matemático simples.

A implementação foi mantida próxima ao que o enunciado solicita. O foco está no funcionamento da lógica e na clareza da organização, não em simular uma infraestrutura energética real com precisão física. O repositório também inclui um notebook Jupyter com o mesmo código, explicado passo a passo.

## 3. Organização dos dados

Para a previsão, o programa guarda dois históricos em listas: `vento_historico = [8, 10, 12]` e `energia_historico = [20, 25, 30]`, este último com a geração eólica. Cada posição de uma lista corresponde à mesma posição da outra: vento de 8 km/h está associado a 20 unidades de energia, por exemplo.

Os subsistemas são representados por um dicionário chamado `sistemas`. A chave `energetico` contém outro dicionário, com `solar` e `eolico` marcados como verdadeiros. As chaves `suporte_vida`, `iluminacao` e `lazer` completam a representação, e todos os sistemas começam ligados. Essa estrutura permite reunir dados relacionados e representar uma hierarquia simples.

```
sistemas = {
    "energetico": {"solar": True, "eolico": True},
    "suporte_vida": True,
    "iluminacao": True,
    "lazer": True
}
```

Os valores informados pelo usuário ficam em um segundo dicionário, chamado `dados`, com as chaves `energia`, `solar`, `eolica`, `geracao`, `consumo` e `vento`. A geração total é a soma da geração solar com a eólica. A prioridade entre os sistemas é definida por duas tuplas: os sistemas essenciais nunca são desligados, e a ordem de corte indica o que é desligado primeiro.

```
ESSENCIAIS = ("energetico", "suporte_vida")
ORDEM_CORTE = ("lazer", "iluminacao")
```

No código atual, o dicionário `sistemas` é alterado pelas regras de decisão: o modo de economia desliga lazer e iluminação, e a operação normal os religa. A opção de menu "Mostrar sistemas" exibe a hierarquia em forma de árvore, com o estado de cada subsistema. Os dados ficam apenas em memória e não são gravados em arquivo.

## 4. Regras de decisão

### 4.1 Comparação entre geração e consumo

A função `analisar_energia(gerac, cons)` compara a geração total com o consumo e retorna a situação encontrada. Quando o consumo supera a geração, o programa mostra um alerta. Se a geração for maior, sugere armazenar o excedente. Quando os valores são iguais, informa que existe equilíbrio. Os valores também são exibidos em barras de texto, para facilitar a comparação.

### 4.2 Verificação de energia disponível

A função `decidir(energ, cons, gerac)` aplica as condições em ordem de prioridade, e a primeira que for verdadeira é executada. Os limites ficam em constantes no início do código: `LIMITE_ENERGIA = 50` e `LIMITE_CONSUMO_ALTO = 70`.

| Prioridade | Condição | Resposta |
|---|---|---|
| 1 | energia < 50 e consumo > 70 | Modo de economia: desliga lazer e iluminação; suporte à vida mantido |
| 2 | energia < 50 | Recomenda reduzir o consumo |
| 3 | consumo > geração (energia normal) | Atenção: a reserva está sendo gasta; reduzir o consumo |
| 4 | demais casos | Operação normal; religa os sistemas desligados pela economia |

A regra combinada usa o operador lógico `and`. Quando ela é aplicada, a função altera o dicionário `sistemas`: lazer e, depois, iluminação são desligados, enquanto `suporte_vida` e `energetico` permanecem ligados por serem essenciais. Em operação normal, os sistemas desligados são religados. É importante distinguir o estado simulado da ação real: o desligamento acontece apenas no estado do programa e nenhum equipamento físico é controlado.

## 5. Previsão de energia eólica

A função `calcular_previsao(vento_alvo)` utiliza `numpy.polyfit` com grau 1 para ajustar uma reta aos pares históricos de vento e geração eólica. Com os dados iniciais, a relação encontrada é energia estimada = 2,5 × vento. Para um vento de 11 km/h, a saída é 27,50 unidades.

```
a, b = np.polyfit(vento_historico, energia_historico, 1)
previsao = max(0.0, a * vento_alvo + b)
```

A previsão nunca fica abaixo de zero, e a função informa quando os dados são insuficientes (menos de dois pontos ou ventos todos iguais). A cada cadastro, o vento e a geração eólica informados entram no histórico, e a reta é recalculada. Antes de incluir o novo ponto, o programa compara o que o modelo previa com o valor observado, para que a previsão nunca use o próprio dado que está sendo testado. Por isso, depois de novos cadastros, o resultado para 11 km/h pode deixar de ser exatamente 27,50. O gráfico é aberto com a biblioteca `matplotlib` e mostra os pontos do histórico, a reta de ajuste e a previsão.

Essa estimativa demonstra a aplicação de regressão linear simples estudada na etapa. Como a amostra inicial contém somente três observações e foi definida para o exercício, o resultado não deve ser interpretado como uma previsão validada da produção eólica real em Marte.

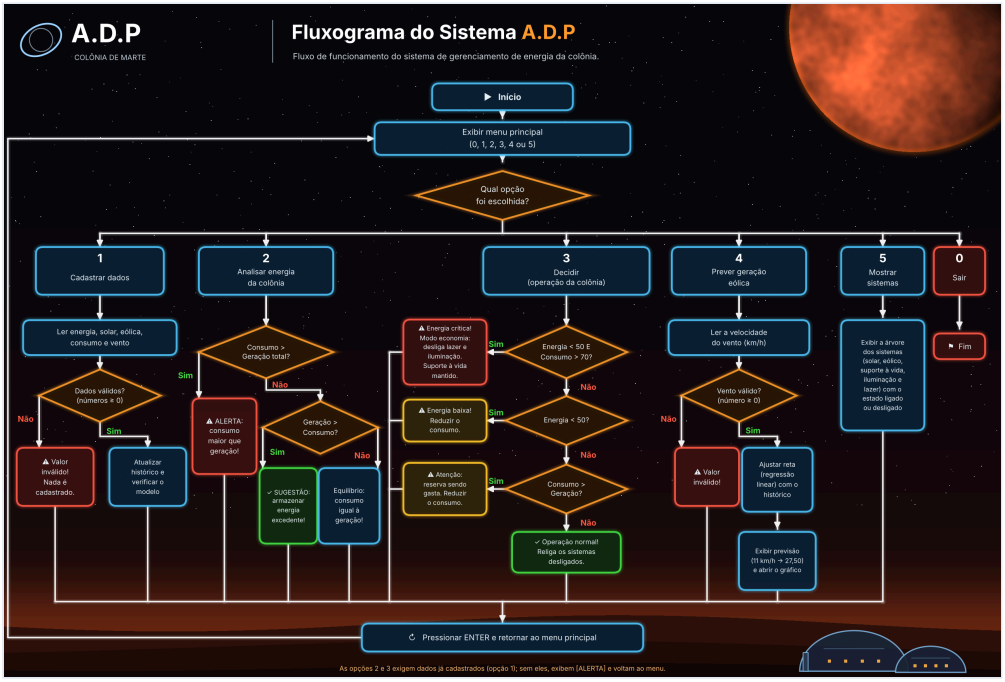
## 6. Funcionamento do programa

O sistema inicia com um menu em repetição contínua. Na opção 1, o usuário cadastra energia disponível, geração solar, geração eólica, consumo e velocidade do vento. As opções 2 e 3 executam, respectivamente, a análise energética e a decisão por regras, e só ficam disponíveis depois do cadastro. A opção 4 pede uma velocidade de vento, mostra o histórico e a estimativa e abre o gráfico da regressão. A opção 5 exibe a árvore dos sistemas e a opção 0 encerra o programa.

O código separa a lógica (funções que apenas calculam e retornam resultados), a exibição (funções que apenas mostram informações) e a entrada de dados. O tratamento de erros usa `try/except ValueError` para entradas que não podem ser convertidas em números e rejeita valores negativos, tanto no cadastro quanto na previsão. Um novo cadastro substitui os dados atuais e acrescenta um ponto ao histórico da regressão, mas nada é salvo entre execuções.

### 6.1 Fluxograma de Funcionamento

Abaixo, o fluxograma ilustra como o sistema processa as informações, analisa a energia e toma as decisões automáticas:




Fluxograma do Sistema A.D.P.

## 7. Exemplos de entrada e saída

Os cenários abaixo foram obtidos executando o programa com as entradas indicadas. As saídas completas, em imagens, estão na seção de prints de execução do README do repositório.

| Situação | Entrada | Resultado |
|---|---|---|
| Consumo acima da geração | solar = 10; eólica = 20; consumo = 80 | [ALERTA] Consumo maior que geração! |
| Excedente energético | solar = 50; eólica = 30; consumo = 30 | [SUGESTÃO] Armazenar energia excedente! |
| Energia crítica e consumo alto | energia = 40; consumo = 80 | Modo de economia: lazer e iluminação desligados; suporte à vida mantido |
| Consumo acima da geração, com energia normal | energia = 60; geração = 30; consumo = 50 | [ATENÇÃO] A reserva está sendo gasta; reduzir o consumo |
| Situação normalizada | energia = 70; geração = 70; consumo = 50 | Operação normal; iluminação e lazer religados |
| Previsão eólica | vento 11 km/h (histórico inicial) | 27,50 unidades de energia |

## 8. Contribuição para a gestão da colônia

O programa permite identificar rapidamente uma situação em que o consumo supera a geração e responde de acordo com a energia disponível: recomenda reduzir o consumo, ativa o modo de economia preservando o suporte à vida e religa os sistemas quando a situação se normaliza. Também mostra como dados históricos podem servir de base para uma estimativa e como comparar a previsão com o valor realmente observado. Na prática, essas funções ajudam a demonstrar a diferença entre observar dados, aplicar regras e produzir previsões.

Por enquanto, o protótipo não considera reservas acumuladas ao longo do tempo, variações climáticas adicionais ou desempenho real dos equipamentos, e os limites de 50 e 70 são fixos no código. O desligamento dos sistemas é uma simulação em memória. Esses pontos seriam possibilidades de evolução, mas não fazem parte da implementação apresentada.

## 9. Considerações finais

O Sistema A.D.P. reúne em um único programa os conceitos centrais pedidos na atividade: listas e dicionários, organização hierárquica, funções, estruturas condicionais com prioridades, análise de geração e consumo e regressão linear simples. O menu facilita a demonstração de cada parte sem exigir que o usuário altere o código, e o notebook acompanha o projeto com explicações para estudo.

O principal resultado é a integração dos conteúdos em um exercício concreto. A implementação também deixa claro o que ainda poderia ser melhorado, como registrar medições em arquivo, permitir a configuração dos limites, ampliar o conjunto de dados da regressão e calcular a reserva de energia ao longo do tempo.

## 10. Referências e materiais consultados


- FIAP. A Aurora Estabelece os Primeiros Sistemas da Colônia. Capítulo 1, 2026. Enunciado e critérios de avaliação.
- FIAP. A Estrutura de Armazenamento Organizando os Dados que Sustentam a Colônia. Capítulo 2, 2026. Lógica booleana.
- FIAP. A Lógica Estruturada que Dá Forma às Primeiras Soluções da Base. Capítulo 3, 2026. Funções e programação estruturada.
- FIAP. As Estruturas Avançadas que Rastreiam Recursos e Informações da Colônia. Capítulo 4, 2026. Estruturas de dados.
- FIAP. Os Vetores e Matrizes que Simulam o Comportamento Inicial da Infraestrutura. Capítulo 5, 2026. Vetores e NumPy.
- FIAP. O Sistema Operacional que Orquestra os Processos da Colônia. Capítulo 6, 2026. Contexto computacional.
- FIAP. A Regressão Inicial que Permite Prever Variáveis da Nova Base. Capítulo 7, 2026. Regressão linear simples.
- FIAP. As Fontes Renováveis que Alimentam os Primeiros Sistemas da Colônia. Capítulo 8, 2026. Energia solar e eólica.
- FRANÇA, Nicolas. Relatório de Atividade de Extensão FIAP (Parte 2 Ajustado). 2026. Material complementar da equipe.


## 11. Repositório público

Repositório público do projeto:
https://github.com/YagoSDev/Sistema_ADP_Colonia_Marte